

Модуль 10. Продакшн, инфраструктура и DevOps

Введение в модуль

Код, который работает на ноутбуке разработчика, ещё не приносит пользы бизнесу. Продакшн — это состояние, в котором приложение живёт самостоятельно: переживает рестарты, сообщает о своём здоровье, масштабируется под нагрузку и обновляется без простоя. Для ML-сервиса это особенно критично: модель занимает гигабайты памяти, inference требует GPU, а задержка в 100 мс может означать разницу между рабочим продуктом и провалом.

В этом модуле мы разберём:

- как упаковать FastAPI-приложение в контейнер так, чтобы образ весил мегабайты, а не гигабайты;
- почему конфигурация через переменные окружения — не модное течение, а необходимость, вытекающая из теории типов;
- что такое observability и почему три столпа (логи, метрики, трейсы) неразделимы;
- как математика очередей определяет число worker'ов Uvicorn;
- как деплоить так, чтобы пользователи не заметили обновления, даже если новая версия модели окажется дефектной.

10.1. Контейнеризация

10.1.1. Что такое контейнер: namespaces и cgroups

Контейнер — это **не виртуальная машина**. VM эмулирует железо: у неё свой kernel, свои драйверы, своя операционная система. Контейнер — это **изолированный процесс** в ядре хоста.

Linux предоставляет два механизма для изоляции:

Namespaces — изоляция пространств имён. Каждый контейнер видит своё:

- **PID namespace**: свой набор процессов (PID 1 — это ваше приложение, а не systemd).
- **NET namespace**: свой сетевой стек, интерфейсы, iptables.

- **MNT namespace:** своё дерево файловых систем.
- **USER namespace:** свою таблицу пользователей (root внутри контейнера может быть UID 100000 снаружи).

cgroups (control groups) — ограничение ресурсов. Ядро отслеживает и лимитирует:

- CPU time (shares, quotas).
- Память (limits, swap).
- I/O диска и сети.
- Число процессов (pids).

Контейнер делится ядром с хостом, поэтому он «легкий»: запуск за миллисекунды, накладные расходы на CPU — единицы процентов, память — только на само приложение, а не на гостевую ОС.

10.1.2. Dockerfile для FastAPI: многоэтапная сборка

Проблема: если в образ установить `gcc`, `python3-dev`, `build-essential` для компиляции зависимостей, они останутся в финальном образе навсегда, хотя нужны были только на этапе сборки. Образ раздувается до 1–2 ГБ.

Multi-stage build решает это: один этап — для сборки, другой — для выполнения. Только артефакты переносятся между этапами.

```
In [1]: # ===== ЭТАП 1: Сборка =====
FROM python:3.12-slim AS builder

WORKDIR /app

# Установка системных зависимостей для компиляции
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Установка Python-зависимостей в виртуальное окружение
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY requirements.txt .
```

```

RUN pip install --no-cache-dir -r requirements.txt

# ===== ЭТАП 2: Выполнение =====
FROM python:3.12-slim AS runtime

WORKDIR /app

# Копируем только виртуальное окружение из builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Создаём непривилегированного пользователя (безопасность)
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
USER appuser

# Копируем код приложения
COPY --chown=appuser:appgroup ./app ./app

# Health check
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Что происходит:

- Этап `builder` устанавливает компиляторы, собирает wheels, создаёт `venv`.
- Этап `runtime` берёт чистый `python:3.12-slim` (без компиляторов) и копирует только готовое `venv`.
- Финальный образ не содержит `gcc`, `apt-cache`, исходников.
- Пользователь `appuser` — не `root`. Если приложение скомпрометировано, злоумышленник не получает `root`-доступ к хосту.

10.1.3. Кэширование слоёв

Docker образ — это набор **слоёв** (layers), каждый из которых — результат инструкции `RUN`, `COPY`, `ADD`. Слои кэшируются: если инструкция и её входные данные не изменились, Docker использует кэш.

Правило: размещайте инструкции от наиболее часто изменяемых к наименее изменяемым.

```
In [2]: # ПЛОХО: requirements.txt копируется ПОСЛЕ кода
COPY ./app ./app          # <- меняется при каждом коммите
COPY requirements.txt .    # <- кэш слоя выше инвалидируется
RUN pip install -r requirements.txt

# ХОРОШО: сначала зависимости (редко меняются), потом код
COPY requirements.txt .
RUN pip install -r requirements.txt # <- кэшируется надолго

COPY ./app ./app          # <- только этот слой пересобирается
```

10.1.4. Оптимизация образа: distroless, slim, размер и безопасность

Базовый образ	Размер	Содержимое	Когда использовать
<code>python:3.12</code>	~900 МБ	Полный Debian, dev-tools	Сборка, отладка
<code>python:3.12-slim</code>	~120 МБ	Debian минимальный	Стандарт для production
<code>python:3.12-alpine</code>	~55 МБ	Alpine Linux, musl	Очень маленький, но musl может ломать бинарные wheels
<code>gcr.io/distroless/python3</code>	~50 МБ	Только Python runtime, без shell, без пакетного менеджера	Максимальная безопасность, сложная отладка

Distroless — образы от Google, содержащие только runtime и зависимости приложения. Нет `bash`, нет `apt`, нет `curl`. Если злоумышленник проникнет в контейнер — ему нечего запустить.

```
In [3]: FROM gcr.io/distroless/python3-debian12
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY ./app ./app
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Недостаток distroless: невозможно зайти в контейнер (`docker exec`) для отладки. Решение — sidecar-контейнер с дебаг-утилитами или использование `debug` -тегов distroless.

10.1.5. Docker Compose: локальная разработка

```
In [4]: # docker-compose.yml
version: "3.9"

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql+asyncpg://user:pass@db:5432/ml_db
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis
    volumes:
      - ./app:/app/app # hot-reload для разработки

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: ml_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  ml-worker:
    build:
      context: .
      dockerfile: Dockerfile.worker
    environment:
      - BROKER_URL=redis://redis:6379
    depends_on:
```

```
- redis

volumes:
  postgres_data:
```

Команды:

```
In [5]: docker-compose up --build      # сборка и запуск
        docker-compose up -d          # в фоне
        docker-compose logs -f api    # логи сервиса api
        docker-compose down -v        # остановка + удаление volumes
```

10.2. Конфигурация и управление средой

10.2.1. Pydantic Settings (pydantic-settings)

В Модуле 4 мы использовали Pydantic для валидации HTTP-запросов. Тот же механизм применим к конфигурации:

```
In [6]: from pydantic_settings import BaseSettings, SettingsConfigDict
        from pydantic import PostgresDsn, RedisDsn, Field

        class Settings(BaseSettings):
            model_config = SettingsConfigDict(
                env_file=".env",
                env_file_encoding="utf-8",
                case_sensitive=False,
            )

            # Обязательные
            database_url: PostgresDsn
            redis_url: RedisDsn
            secret_key: str = Field(min_length=32)

            # С значениями по умолчанию
            debug: bool = False
            log_level: str = "INFO"

            # Числовые с валидацией
            max_workers: int = Field(default=4, ge=1, le=64)
            access_token_expire_minutes: int = Field(default=15, ge=1)
```

```
# Префикс для переменных окружения (опционально)
model_config = SettingsConfigDict(env_prefix="APP_")

settings = Settings()
```

Если переменная окружения `DATABASE_URL` отсутствует или не является валидным PostgreSQL URL — приложение **не запустится** при импорте модуля. Это fail-fast: ошибка конфигурации обнаруживается на старте, а не в 3 часа ночи в продакшене.

10.2.2. Разделение конфигураций: dev, staging, production

Антипаттерн: ветвление в коде `if env == "prod": ... else: ...`. Это не масштабируется и приводит к «prod-only» багам.

Правильно: одна кодовая база, разные значения переменных окружения.

```
In [7]: # .env.development
DEBUG=true
LOG_LEVEL=DEBUG
DATABASE_URL=postgresql+asyncpg://localhost/dev_db

# .env.production
DEBUG=false
LOG_LEVEL=WARNING
DATABASE_URL=postgresql+asyncpg://prod-cluster/db
```

Загрузка:

```
In [8]: import os

env = os.getenv("ENV", "development")
settings = Settings(_env_file=f".env.{env}")
```

10.2.3. The Twelve-Factor App

Манифест от Heroku (2011), описывающий лучшие практики SaaS-приложений. Ключевые пункты для конфигурации:

III. Config: Store config in the environment Конфигурация строго в переменных окружения. Никаких `config.py` с жёстко зашитыми паролями, никаких `if hostname == 'prod-server'`.

Почему?

- Одна кодовая база может быть развёрнута в любой среде без изменений.
- Конфигурация может меняться без redeploy (перезапуск процесса достаточен).
- Уменьшает риск коммита секретов в Git.

V. Build, release, run: Strictly separate build and run stages

- **Build:** компиляция, установка зависимостей, создание Docker-образа. Результат — immutable artifact.
- **Release:** объединение build-артефакта с конфигурацией (env vars). Результат — runnable unit.
- **Run:** запуск процессов. Код не меняется.

Это означает: если нужно изменить `LOG_LEVEL` — не пересобирайте образ, просто измените env и перезапустите контейнер.

10.2.4. Хранение секретов

Переменные окружения — хороший минимум, но не идеал для высокочувствительных данных (приватные ключи, API-ключи платёжных систем).

Иерархия:

Уровень	Пример	Когда
1. Env vars	<code>SECRET_KEY=abc</code>	Некритичные секреты, development
2. Docker secrets	<code>/run/secrets/db_password</code>	Docker Swarm, простые кластеры
3. Kubernetes secrets	<code>kubect1 create secret</code>	K8s
4. Vault	HashiCorp Vault, AWS Secrets Manager, Azure Key Vault	Enterprise, динамические секреты

HashiCorp Vault: приложение аутентифицируется в Vault (через token, Kubernetes SA или AWS IAM) и получает временные учётные данные с TTL. Даже при компрометации контейнера злоумышленник получает доступ только до истечения TTL.

```
In [9]: # Псевдокод для Vault
import hvac

client = hvac.Client(url="https://vault.example.com")
client.auth.kubernetes.login(role="ml-api", jwt=service_account_token)

secret = client.secrets.kv.v2.read_secret_version(path="database/creds")
db_password = secret["data"]["data"]["password"] # временный, живёт 1 час
```

10.3. Логирование, метрики, трассировка (Observability)

Observability — способность понимать внутреннее состояние системы по её внешним выходным данным. Три столпа:

Столп	Что отвечает	Пример вопроса
Логи (Logs)	Что произошло в конкретный момент	«Почему запрос от пользователя 123 упал в 14:32?»
Метрики (Metrics)	Как система ведёт себя агрегированно	«Какой p95 latency за последний час?»
Трейсы (Traces)	Как запрос проходит через распределённую систему	«Где именно потерялось 800 мс — в БД или в ML-модели?»

10.3.1. Структурированные логи: structlog , JSON-формат

Традиционные логи — неструктурированный текст:

```
In [10]: 2024-01-15 14:32:01 INFO User alice logged in
```

Это сложно парсить. Структурированные логи — JSON:

```
In [11]: {
  "timestamp": "2024-01-15T14:32:01Z",
  "level": "info",
  "event": "user_login",
  "user_id": "alice",
  "ip": "192.168.1.1",
```

```
"request_id": "req-abc123"
}
```

structlog — библиотека для структурированного логирования в Python:

```
In [12]: import structlog
import uuid
from fastapi import Request
from contextvars import ContextVar

correlation_id: ContextVar[str] = ContextVar("correlation_id", default="unknown")

def get_logger():
    return structlog.get_logger(
        request_id=correlation_id.get(),
        service="ml-api",
    )

logger = get_logger()

# Использование
logger.info("prediction_made", model_version="v2", latency_ms=45, user_id="123")
```

Correlation ID: уникальный идентификатор запроса, который «приклеивается» ко всем логам в рамках одного запроса и передаётся между микросервисами (через HTTP-заголовок `X-Request-ID` или `traceparent`).

```
In [13]: @app.middleware("http")
async def add_correlation_id(request: Request, call_next):
    req_id = request.headers.get("x-request-id", str(uuid.uuid4()))
    correlation_id.set(req_id)

    response = await call_next(request)
    response.headers["x-request-id"] = req_id
    return response
```

Без correlation ID поиск по логам в распределённой системе превращается в кошмар: сотни сервисов, тысячи переплетающихся запросов.

10.3.2. Метрики: Prometheus + Grafana

Prometheus — система сбора метрик по pull-модели. Периодически (обычно каждые 15 секунд) забирает метрики с `/metrics` endpoint'a приложения.

Типы метрик Prometheus:

Тип	Смысл	Пример
Counter	Монотонно растущее значение	Число предсказаний, число ошибок
Gauge	Текущее значение, может расти и падать	Размер очереди, использование памяти
Histogram	Распределение значений по корзинам (buckets)	Latency inference, размер запроса
Summary	Похоже на Histogram, но считает квантили на стороне клиента	p95 latency (устареваает, Histogram предпочтительнее)

Инструментирование FastAPI:

```
In [14]: from prometheus_client import Counter, Histogram, generate_latest, CONTENT_TYPE_LATEST
from fastapi import Response

# Определение метрик
PREDICTION_COUNTER = Counter(
    "ml_predictions_total",
    "Total predictions",
    ["model_version", "status"] # лейблы для сегментации
)

PREDICTION_LATENCY = Histogram(
    "ml_prediction_duration_seconds",
    "Prediction latency",
    ["model_version"],
    buckets=[0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0]
)

@app.post("/predict")
async def predict(input_data: InputModel):
    start = time.time()
    try:
        result = await model_service.predict(input_data)
```

```

        PREDICTION_COUNTER.labels(model_version="v2", status="success").inc()
    return result
except Exception:
    PREDICTION_COUNTER.labels(model_version="v2", status="error").inc()
    raise
finally:
    PREDICTION_LATENCY.labels(model_version="v2").observe(time.time() - start)

@app.get("/metrics")
async def metrics():
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)

```

Grafana — визуализация метрик. Запросы на языке PromQL:

```

In [15]: # RPS за последние 5 минут
rate(ml_predictions_total[5m])

# p95 latency
histogram_quantile(0.95, rate(ml_prediction_duration_seconds_bucket[5m]))

# Ошибки в процентах
rate(ml_predictions_total{status="error"}[5m])
/ rate(ml_predictions_total[5m])

```

10.3.3. Математическая подоплека: гистограммы и квантили

Prometheus Histogram хранит не отдельные значения, а **кумулятивные счётчики по корзинам** (buckets). Для latency с корзинами `[0.01, 0.05, 0.1, 0.5, 1.0, +Inf]` :

Bucket	Значение (cumulative)
≤ 0.01	100
≤ 0.05	350
≤ 0.1	800
≤ 0.5	950
≤ 1.0	990
$\leq +Inf$	1000

Чтобы найти p95 (950-й элемент из 1000):

- 950 попадает в корзину ≤ 0.5 (там 950 наблюдений).
- Значит $p_{95} \leq 0.5$ с.

Формула `histogram_quantile(φ, buckets)` использует **линейную интерполяцию** внутри корзины:

$$\text{quantile} = b_{k-1} + \frac{\phi \cdot N - c_{k-1}}{c_k - c_{k-1}} \cdot (b_k - b_{k-1})$$

где:

- b_k — верхняя граница корзины,
- c_k — кумулятивный счётчик в корзине k ,
- N — общее число наблюдений.

Проблема: если p95 попадает в корзину `[0.1, 0.5]`, мы знаем только, что $p_{95} \in [0.1, 0.5]$.

Точное значение неизвестно. Поэтому важно выбирать buckets под ожидаемое распределение: для ML API, где большинство запросов < 100 мс, но иногда бывают выбросы до 5 секунд, нужны корзины `[0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0]`.

10.3.4. Распределённая трассировка: OpenTelemetry, Jaeger

Когда запрос проходит через API Gateway -> Auth Service -> FastAPI -> ML Server -> PostgreSQL, где теряется время? Логи показывают отдельные события, метрики — агрегаты.

Трейс показывает полный путь.

OpenTelemetry — стандарт для сбора телеметрии (логи, метрики, трейсы). **Trace** состоит из **спанов** (spans):

```
In [16]: Trace: predict-request-123
|─ Span: api-gateway (0ms - 5ms)
|─ Span: auth-service (5ms - 15ms)
|─ Span: fastapi-predict (15ms - 120ms)
|   |─ Span: db-query (20ms - 35ms)
|   |─ Span: redis-cache (35ms - 36ms, cache miss)
|   └─ Span: ml-inference (40ms - 110ms)
└─ Span: postprocess (110ms - 120ms)
```

Каждый спан имеет:

- `trace_id` — идентификатор всего трейса.

- `span_id` — идентификатор спана.
- `parent_id` — ссылка на родительский спан.
- `start_time` , `end_time` .
- Атрибуты (например, `model_version` , `input_size`).

Интеграция с FastAPI:

```
In [17]: from opentelemetry import trace
from opentelemetry.exporter.jaeger.thrift import JaegerExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.sqlalchemy import SQLAlchemyInstrumentor

# Настройка
trace.set_tracer_provider(TracerProvider())
tracer = trace.get_tracer(__name__)

jaeger_exporter = JaegerExporter(
    agent_host_name="jaeger",
    agent_port=6831,
)
trace.get_tracer_provider().add_span_processor(
    BatchSpanProcessor(jaeger_exporter)
)

# Инструментирование FastAPI и SQLAlchemy
FastAPIInstrumentor.instrument_app(app)
SQLAlchemyInstrumentor().instrument()

# Ручной спан
@app.post("/predict")
async def predict(input_data: InputModel):
    with tracer.start_as_current_span("ml_inference") as span:
        span.set_attribute("model.version", "v2")
        span.set_attribute("input.features_count", len(input_data.features))
        result = await model_service.predict(input_data)
    return result
```

Trace Context Propagation: когда FastAPI вызывает ML Server по HTTP, `trace_id` передаётся через заголовок `tracetransparent` (стандарт W3C). ML Server продолжает тот же трейс, добавляя свои спаны.

10.3.5. Health checks: liveness, readiness

Kubernetes (и другие оркестраторы) используют health checks для управления жизненным циклом подов.

Проверка	Что означает	Действие при провале
Liveness	«Приложение живо, не зависло»	Перезапуск контейнера (kill + recreate)
Readiness	«Приложение готово принимать трафик»	Удаление из балансировки (не отправлять запросы)
Startup	«Приложение завершило инициализацию»	Ждать, не проверять liveness/readiness

```
In [18]: from fastapi import FastAPI

app = FastAPI()

@app.get("/health/live")
async def liveness():
    # Просто проверяем, что процесс не завис
    return {"status": "alive"}

@app.get("/health/ready")
async def readiness():
    # Проверяем критичные зависимости
    if not await db.is_connected():
        raise HTTPException(status_code=503, detail="Database unavailable")
    if not await redis.ping():
        raise HTTPException(status_code=503, detail="Redis unavailable")
    return {"status": "ready"}
```

Зачем разделять?

- При деплое новой версии: startup + readiness не проходят, пока модель не загрузилась. Kubernetes не отправляет трафик.
- Если приложение зависло (deadlock в event loop): liveness падает, Kubernetes перезапускает контейнер.
- Если база данных временно недоступна: readiness падает, трафик уходит на другие поды, но текущий под не убивают (БД скоро восстановится).

10.4. Масштабирование

10.4.1. Горизонтальное vs вертикальное

Вертикальное масштабирование: увеличение ресурсов одного сервера (больше CPU, RAM, GPU). Просто, но есть предел (физический сервер, стоимость).

Горизонтальное масштабирование: добавление серверов. Требует:

- **Stateless** приложения (нет локального состояния между запросами).
- **Балансировщика нагрузки** (Nginx, Traefik, AWS ALB, Kubernetes Ingress).
- **Shared storage** для состояния (Redis, PostgreSQL, S3).

FastAPI-приложение по своей природе stateless (если не хранить сессии в памяти). Это идеально для горизонтального масштабирования.

10.4.2. Gunicorn + Uvicorn workers

Uvicorn — ASGI-сервер. Но он однопроцессный. Для использования всех ядер CPU запускают **Gunicorn** как process manager с Uvicorn worker'ами.

```
In [19]: gunicorn app.main:app \
        -w 4 \
        -k uvicorn.workers.UvicornWorker \
        --bind 0.0.0.0:8000 \
        --access-logfile - \
        --error-logfile - \
        --timeout 120
```

Параметр	Значение	Смысл
<code>-w 4</code>	4 worker-процесса	Каждый worker — отдельный процесс с event loop
<code>-k uvicorn.workers.UvicornWorker</code>	Класс worker'a	Uvicorn внутри Gunicorn
<code>--timeout 120</code>	120 секунд	Worker, не отвечающий 120 секунд, считается зависшим и перезапускается

Сколько worker'ов?

Для **синхронных** worker'ов (Django, Flask) классическая формула:

$$N_{workers} = 2 \cdot C + 1$$

где C — число CPU-ядер. Это учитывает, что пока один worker ждёт I/O, другой использует CPU.

Для **асинхронных** worker'ов (Uvicorn) ситуация иная: один процесс с event loop обслуживает тысячи конкурентных соединений. Дополнительные процессы нужны только для:

- Использования всех ядер CPU (Python GIL ограничивает один поток).
- Отказоустойчивости (если один процесс упал, остальные работают).
- Параллелизма CPU-bound задач (если используется `ProcessPoolExecutor` внутри).

Рекомендация для FastAPI:

$$N_{workers} = C$$

Иногда $C + 1$, но не $2C + 1$ — асинхронные worker'ы не тратят время на ожидание в том же смысле, что синхронные.

Uvicorn без Gunicorn (для контейнеров):

```
In [20]: uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

В контейнере Kubernetes часто проще запускать один процесс Uvicorn на под и масштабировать количество подов, а не worker'ов внутри пода. Это даёт лучшую гранулярность для Horizontal Pod Autoscaler.

10.4.3. Масштабирование WebSocket: sticky sessions, Redis Pub/Sub

WebSocket-соединение — это **stateful** долгоживущее TCP-соединение. Если клиент подключился к серверу А, а следующий запрос (в рамках того же WebSocket) ушёл на сервер В через балансировщик — связь разорвётся.

Решение 1: Sticky Sessions (Session Affinity)

Балансировщик «приклеивает» клиента к конкретному серверу по хешу IP или cookie.

```
In [21]: # Nginx
         upstream websocket_backend {
             ip_hash; # sticky по IP
             server ws1:8000;
```

```
server ws2:8000;  
}
```

Проблема: если сервер падает, все его WebSocket-клиенты отключаются. Нет failover.

Решение 2: Redis Pub/Sub для broadcast

Если нужно отправить сообщение всем клиентам (например, broadcast о завершении обучения модели), а клиенты разбросаны по разным серверам:

In [22]:

```
Клиент A → Server 1 ─┐  
                        │→ Redis Pub/Sub ← Server 3 ← Клиент C  
Клиент B → Server 2 ─┘
```

In [23]:

```
import redis.asyncio as redis  
  
redis_client = redis.Redis()  
  
@app.websocket("/ws/notifications")  
async def notifications(websocket: WebSocket):  
    await websocket.accept()  
  
    pubsub = redis_client.pubsub()  
    await pubsub.subscribe("model_updates")  
  
    async for message in pubsub.listen():  
        if message["type"] == "message":  
            await websocket.send_text(message["data"])
```

Когда модель обновляется, любой сервер публикует:

In [24]:

```
await redis_client.publish("model_updates", json.dumps({"event": "model_retrained"}))
```

Все подписанные WebSocket-соединения на всех серверах получают уведомление.

10.4.4. Математическая подоплека: закон Литтла и capacity planning

Закон Литтла (вспомним Модуль 8):

$$L = \lambda W$$

где:

- L — среднее число одновременно обрабатываемых запросов.
- λ — интенсивность поступления (RPS).
- W — среднее время обработки (latency в секундах).

Пример: ML API обрабатывает $\lambda = 500$ RPS, средняя latency $W = 0.2$ с.

$$L = 500 \cdot 0.2 = 100$$

В системе в среднем 100 одновременных запросов. Если каждый worker (процесс Uvicorn) может обслужить C_{worker} конкурентных соединений (ограничено памятью и CPU), то нужно:

$$N_{workers} \geq \frac{L}{C_{worker}}$$

Если один worker на 1 CPU + 2 ГБ RAM держит 500 конкурентных корутин, а у нас $L = 100$ — достаточно 1 worker'а на ядро. Но с запасом на пики: $N_{pods} = \frac{L_{peak}}{C_{worker} \cdot utilization_target}$.

При target utilization 70% и $L_{peak} = 300$:

$$N_{pods} = \frac{300}{500 \cdot 0.7} \approx 1$$

Но с запасом на отказ одного пода и неравномерность нагрузки: 2–3 пода.

10.5. CI/CD

10.5.1. GitHub Actions / GitLab CI

CI (Continuous Integration): автоматическая сборка и тестирование при каждом коммите.

CD (Continuous Delivery/Deployment): автоматический деплой после успешных тестов.

GitHub Actions

```
In [25]: # .github/workflows/ci.yml
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
```

```
test:
  runs-on: ubuntu-latest

  services:
    postgres:
      image: postgres:16-alpine
      env:
        POSTGRES_USER: test
        POSTGRES_PASSWORD: test
        POSTGRES_DB: test_db
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

    redis:
      image: redis:7-alpine
      ports:
        - 6379:6379

  steps:
    - uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.12"

    - name: Install dependencies
      run: |
        pip install uv
        uv pip install --system -r requirements.txt -r requirements-dev.txt

    - name: Lint
      run: |
        ruff check .
        ruff format --check .
        mypy app/

    - name: Run migrations
      env:
        DATABASE_URL: postgresql+asyncpg://test:test@localhost/test_db
      run: alembic upgrade head
```

```

- name: Test
  env:
    DATABASE_URL: postgresql+asyncpg://test:test@localhost/test_db
    REDIS_URL: redis://localhost:6379
  run: pytest --cov=app --cov-report=xml

- name: Upload coverage
  uses: codecov/codecov-action@v3
  with:
    files: ./coverage.xml

build:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Build Docker image
      run: |
        docker build -t ml-api:${{ github.sha }} .
        docker tag ml-api:${{ github.sha }} ml-api:latest

    - name: Push to registry
      run: |
        echo ${ secrets.DOCKER_PASSWORD } | docker login -u ${ secrets.DOCKER_USERNAME } --password-stdin
        docker push ml-api:${{ github.sha }}

```

10.5.2. Стратегии деплоя

Rolling Deployment

Постепенная замена старых подов на новые. Kubernetes по умолчанию:

```

In [26]: strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1      # можно создать 1 под сверх лимита
    maxUnavailable: 0 # нельзя убирать старые поды, пока новые не ready

```

Плюс: простота, нет простоя. **Минус:** если новая версия дефектна, часть пользователей получит ошибки до отката.

Blue-Green Deployment

Два идентичных окружения: Blue (текущее), Green (новое). Трафик переключается мгновенно.

In [27]: Этап 1: [LB] → Blue (v1) Green (v2) — простаивает
 Этап 2: [LB] → Green (v2) Blue (v1) — на случай отката

Плюс: мгновенный откат, ноль простоя. **Минус:** требует вдвое больше ресурсов.

Canary Deployment

Новая версия получает небольшой процент трафика (например, 5%). Мониторинг метрик. Если всё хорошо — постепенно увеличиваем до 100%.

In [28]: Этап 1: 95% → v1
 5% → v2

 Этап 2: 50% → v1
 50% → v2

 Этап 3: 0% → v1
 100% → v2

Canary analysis: автоматическое сравнение метрик v1 и v2 (error rate, latency). Если v2 хуже по p95 более чем на 10% — автоматический откат.

Математика canary: это форма **A/B тестирования** с непрерывными метриками. Для бинарных метрик (error rate) используется **Z-тест для пропорций**:

$$Z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1 - \hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$$

где $\hat{p}_1, \hat{p}_2$ — доли ошибок в **canary baseline**, $\hat{p}$ — общая доля ошибок. Если $|Z| > Z_{\alpha/2}$ (обычно **1.96** для $\alpha = 0.05$), различие статистически значимо — canary откатывается.

10.5.3. Базы данных в CI/CD

Правило: миграции применяются **до** деплоя новой версии приложения. Никогда после.

Почему: если новый код ожидает колонку `email_verified`, а миграция ещё не выполнена — приложение упадёт с `ProgrammingError` при первом же запросе. Если применить миграцию

после деплоя — старый код, который ещё работает на части инстансов, может сломаться (например, удаление колонки, которую он читает).

Expand/Contract в CI/CD:

In [29]:

```
Шаг 1: Deploy миграции (Expand)
- Добавить новую колонку (nullable)
- Добавить новый индекс (concurrently)
- Старый код игнорирует новую колонку – работает

Шаг 2: Deploy новой версии приложения
- Новый код пишет в старую и новую колонку
- Читает из новой колонки, если она не null, иначе из старой

Шаг 3: Backfill данных (фоновый процесс)
- Заполняем новую колонку для существующих строк

Шаг 4: Deploy миграции (Contract)
- Делаем новую колонку NOT NULL
- Удаляем старую колонку
- Удаляем fallback-логику из кода
```

Совместимые изменения (backward-compatible):

- Добавление колонки (`NULL` по умолчанию).
- Добавление индекса (`CONCURRENTLY` в PostgreSQL, чтобы не блокировать таблицу).
- Добавление таблицы.
- Переименование колонки: **не совместимо**. Нужно сначала добавить новую, скопировать данные, переключить код, удалить старую.

Индексы без блокировок:

In [30]:

```
-- Блокирует таблицу на запись (ПЛОХО для production)
CREATE INDEX idx_users_email ON users(email);

-- Не блокирует (ХОРОШО)
CREATE INDEX CONCURRENTLY idx_users_email ON users(email);
```

Alembic по умолчанию генерирует `CREATE INDEX`. Для production нужно переопределить:

```
In [31]: # alembic/versions/xxx_add_index.py
from alembic import op

def upgrade():
    op.create_index(
        'idx_users_email',
        'users',
        ['email'],
        postgresql_concurrently=True, # <- без блокировки
    )
```

Rollback стратегия: каждая миграция должна иметь `downgrade()` . Если деплой новой версии провалился — откатываем код, но **не откатываем миграции** (новый код уже не работает, старый код должен быть совместим с новой схемой). Откат миграций — крайняя мера, только если миграция сломала данные.

Итог модуля